

→ unsigned

Numbers
integer

'int' +ive
-ive

Short
unsigned Short

- 0 +

unsigned int only +ive

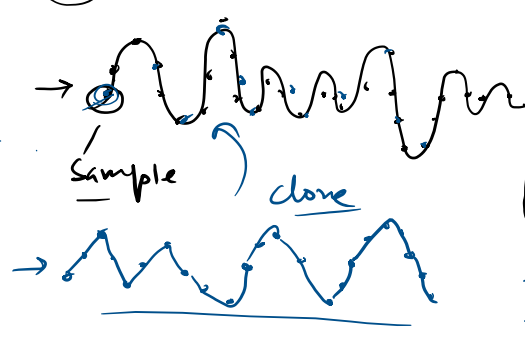
char → 2^8 256 0-255
unsigned char 0-255 +ive

0, 1
digital

float
32, bit float

Voice data

.wav
Signed Analog → digital



quality sampling rate
8000 Hz
→ sample per sec
48000
→ quality
→ data size

.wav
compressed
mp3 short data small
.Ape

1-D dynamic Arrays → 2D dynamic Arrays

Array of pointers: Static (on stack)

int * ptr;

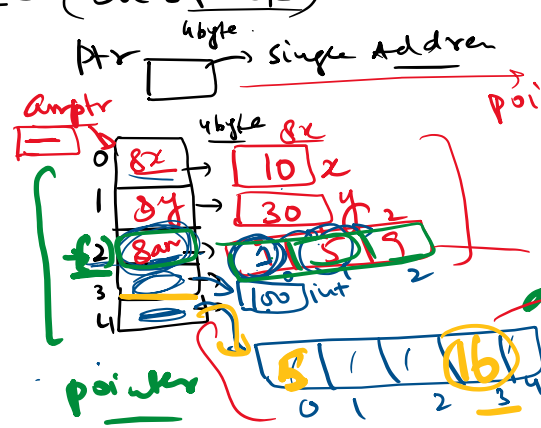
int * arr_ptr[5];

int x = 60;

int y = 30;

arr_ptr[0] = 8x;

arr_ptr[1] = 8y;



points to single element
base address of array
stack / heap

int arr[3] = {1, 5, 9};

arr_ptr[2] = arr;

arr_ptr[3] = new int(100);

arr_ptr[4] = new int[5];

→ delete arr_ptr[3];
→ delete arr_ptr[4];
cout << arr_ptr[3];

*(arr_ptr[3]); value 100 offset

*(arr_ptr[4] + 0) = 5;

arr_ptr[4][0] = 5;

arr_ptr[4][3] = 16;

cout << *(arr_ptr[0]); 10

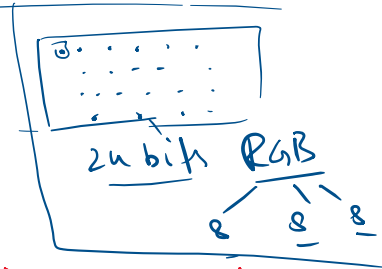
cout << *(arr_ptr[2]); 1

cout << arr_ptr[2][1]; 5

cout << *(arr_ptr[2] + 1); 5

*(arr_ptr[2] + 0) 1
*(arr_ptr[2] + 1) 5
*(arr_ptr[2] + 2) 9

Subscript
arr_ptr[2][0] 1
arr_ptr[2][1] 5
arr_ptr[2][2] 9



2D dynamic Arrays:

1 - An array of pointer $\rightarrow$ Stack $\rightarrow$ partially dynamic 2D array.
 $\rightarrow$ heap $\rightarrow$ completely dynamic array.

2 $\rightarrow$ Store 1d array at each index of arrays of pointers.

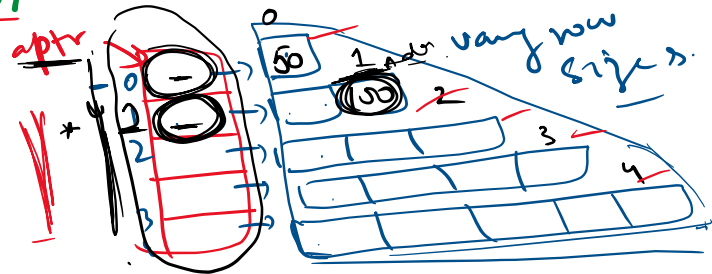
```
const int s = 5;
int * a_ptr[s] = {0};
```

```
a {
    a_ptr[0] = new int[5];
    a_ptr[1] = new int[5];
    a_ptr[2] = new int[s];
    ...
}
```

loop $\rightarrow$ for (int i=0; i < s; i++)
 $\rightarrow$ $\quad \quad \quad$ a_ptr[i] = new int[5];

5x5 number of pointer = num of rows.

$\rightarrow$ 5 1d Arrays created on vary heap.



rows: Array of pointer
 Array of data
 0-1-2-3-4
 $\rightarrow$ $\quad \quad \quad$ a_ptr[0][0] = 50;

$\ast(a_ptr[0] + 0) = 50;$

$\rightarrow$ a_ptr[0][1] = 100; $\times \ast(\ast a_ptr + 0) = 50$

$\rightarrow$ a_ptr[1][1] = 100; $\ast(a_ptr[1] + 1) = 100;$

$\ast(a_ptr + 1)[1] = 100$
 row: array of pointer
 element: data

$\ast(\ast(a_ptr + 1) + 1) = 100;$
 off first pointer

$\ast(a_ptr[0] + 0) = 50;$
 unary: 52+2 52
 binary

2D Arrays 5x5

```
for (int i=0; i < s; i++)
    for (int j=0; j < s; j++)
        {
            cin >> a_ptr[i][j];
            cout << a_ptr[i][j];
        }
```

$\rightarrow$ row wise column wise diagonal wise

Deallocate Dynamic portion.

$\begin{cases} \text{delete } [] \text{ } \text{aptr}[0]; \\ \text{delete } [] \text{ } \text{aptr}[1]; \\ \vdots \end{cases} \rightarrow \begin{cases} \text{for (int i=0; i<s; i++)} \\ \text{delete } [] \text{ } \text{aptr}[i]; \end{cases}$

Arrays of pointers.

→ compile time

→ we cannot resize or change array of pointers. (Number of rows cannot be changed)

→ we can change columns → ^{different} same size

move Array of pointers to heap:

→ multiple indirection: → (store address of one pointer in another pointer)

$\text{int } x = 10;$
 $\text{int } * \text{ptr} = \&x;$

$\text{cout} << x << * \text{p};$
direct indirect

$\text{int } * * \text{dp} = \&\text{p};$
2d-pointer int*

$\text{cout} << \text{dp} << * \text{dp} << ** \text{dp};$
8p 8x x 10

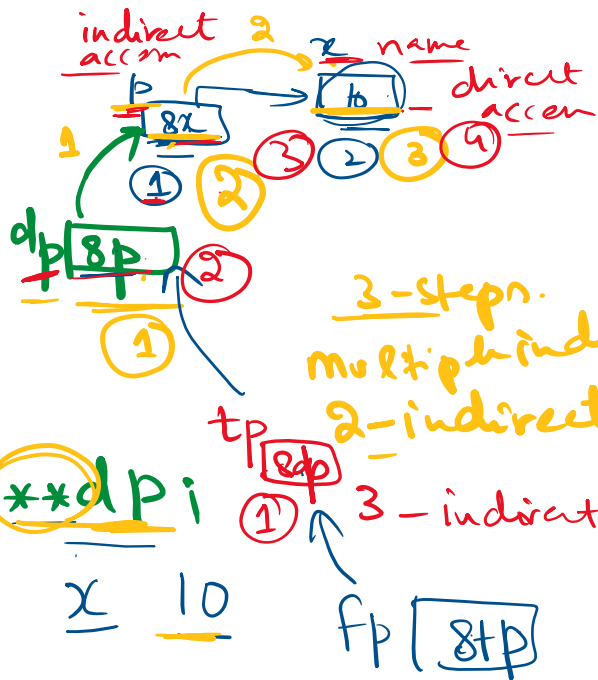
$\text{int } * * * \text{tp} = \&\text{dp};$
int**

$\text{cout} << \text{tp} << * \text{tp} << ** \text{tp} << *** \text{tp};$

$\text{int } * * * * \text{fp} = \&\text{tp};$
int****

$\text{cout} << \text{fp} << * \text{fp} << ** \text{fp} << *** \text{fp} << **** \text{fp};$
8dp 8p 8x x 10 8dp 8p 8x

$\text{cout} << \text{fp} << * \text{fp} << ** \text{fp} << *** \text{fp} << **** \text{fp};$
8dp 8p 8x x 10



3-steps.
multiple indirect
2-indirection
3-indirect